

Diseño de una base de datos de telemetría y experiencia de usuario para la investigación de Chocolate-Doom

(ENTREGA FINAL)



Clase: Bases de Datos

5/06/2026

Maestro: Andrés Oswaldo Calderón Ph.D.

1st Isabella Rodríguez Posada
Estudiante Ing. De Sistemas
Pontificia Universidad Javeriana
Bogotá, Colombia

2nd Juan David Botero Avila
Estudiante Ing. De Sistemas
Pontificia Universidad Javeriana
Bogotá, Colombia

3rd Juan Esteban Ocampo Arocha
Estudiante Ing. De Sistemas
Pontificia Universidad Javeriana
Bogotá, Colombia

4th Miguel Ángel Gómez López
Estudiante Ing. De Sistemas
Pontificia Universidad Javeriana

Bogotá, Colombia

Resumen: Este proyecto tuvo como objetivo el diseño e implementación de una base de datos relacional para el almacenamiento, procesamiento y análisis de telemetría obtenida de partidas del videojuego DOOM, complementada con información demográfica y datos de experiencia de usuario (UX). A lo largo de las tres entregas se desarrolló un modelo conceptual y lógico de la base de datos, se implementó la estructura física en PostgreSQL, se construyó un proceso ETL para la carga de datos y se desarrollaron consultas analíticas, índices y vistas para facilitar el análisis de la información. El resultado es una solución capaz de almacenar grandes volúmenes de datos de telemetría y generar información útil sobre el comportamiento y la interacción de los jugadores.

1. INTRODUCCIÓN

El análisis de datos provenientes de videojuegos se ha convertido en una herramienta importante para comprender el comportamiento de los usuarios, identificar patrones de interacción y evaluar la experiencia de juego. En este proyecto se abordó el problema del almacenamiento y análisis de datos de telemetría generados por una versión modificada de DOOM, capaz de registrar información detallada sobre los movimientos y acciones de los jugadores durante una partida.

Para ello se diseñó una base de datos relacional que integra información de usuarios, jugadores, partidas, mapas, sectores, eventos de telemetría y encuestas UX. El desarrollo se llevó a cabo de manera progresiva en tres etapas: diseño conceptual y lógico, implementación y carga de datos, y finalmente análisis y optimización mediante consultas e índices. Este proceso permitió aplicar conceptos fundamentales de modelado de datos, normalización, integridad referencial, ETL y optimización de consultas en un caso de estudio real.

2. Primera Entrega: Diseño Conceptual y Lógico

2.1 Objetivo

El objetivo de la primera entrega fue diseñar la estructura conceptual y lógica de la base de datos que soportaría el almacenamiento y análisis de la telemetría generada por partidas del videojuego DOOM. Para ello se identificaron las entidades principales del dominio, las relaciones entre ellas, las reglas de negocio, los requisitos funcionales y las consideraciones éticas necesarias para el manejo de la información de los participantes.

2.2 Supuestos y Consideraciones Éticas

Antes de diseñar la base de datos se definieron varios supuestos para delimitar el alcance del proyecto. Se asumió que cada usuario debía otorgar consentimiento para participar en el estudio, que un usuario podría estar asociado a uno o más jugadores dentro del sistema y que cada partida podría involucrar múltiples participantes. Asimismo, se consideró que toda la información de telemetría provendría de archivos generados por una versión instrumental del videojuego DOOM.

Desde el punto de vista ético, se decidió anonimizar la identidad de los participantes mediante códigos anónimos y almacenar únicamente la información necesaria para los objetivos académicos del proyecto. Además, se establecieron mecanismos para garantizar la integridad, trazabilidad y confidencialidad de los datos recopilados. **(Ya explicados en el documento de la primera entrega)**

1. Consentimiento Informado
2. Anonimato de la Información
3. Minimización de Datos
4. Control de Acceso
 - a. Investigador responsable
 - b. Analista
 - c. Administrador Técnico
5. Uso responsable de los datos
6. Prevención de interpretaciones injustas
7. Seguridad y conservación de los datos
8. Calidad y trazabilidad de los datos

2.3 Identificación de Entidades

Durante esta etapa se identificaron las entidades principales que conforman el modelo de datos.

Usuarios y Jugadores

La entidad Usuario almacena la información básica de los participantes, incluyendo datos demográficos, experiencia previa y consentimiento informado. A partir de esta entidad se deriva la entidad Jugador, que representa la identidad utilizada dentro de las partidas del videojuego.

Partidas y Participaciones

La entidad Partida almacena la información general de cada sesión de juego, incluyendo fechas de inicio y fin, modo de juego y configuración utilizada. Debido a que una partida puede tener múltiples jugadores y un jugador puede participar en múltiples partidas, se implementó la entidad intermedia “Participante_Partida”, que resuelve esta relación muchos a muchos.

Episodios, Mapas y Sectores

La estructura espacial del juego se modeló mediante tres entidades jerárquicas:

- Episodio, que agrupa mapas relacionados.
- Mapa, que pertenece a un episodio específico.
- Sector, que representa áreas concretas dentro de cada mapa.

Esta organización permite realizar análisis espaciales sobre los movimientos de los jugadores.

Telemetría

La entidad “Evento_Telemetría” constituye el núcleo del proyecto. En ella se registran las coordenadas del jugador, velocidades, dirección de visión, salud, armadura y demás variables capturadas durante la partida. Cada evento está asociado a una partida específica y puede relacionarse con un jugador y un sector determinado.

Instrumentos UX

Para evaluar la experiencia de usuario se diseñó un conjunto de entidades compuesto por:

- Instrumento_UX
- Pregunta_UX

- Respuesta_UX
- Detalle_Respuesta_UX

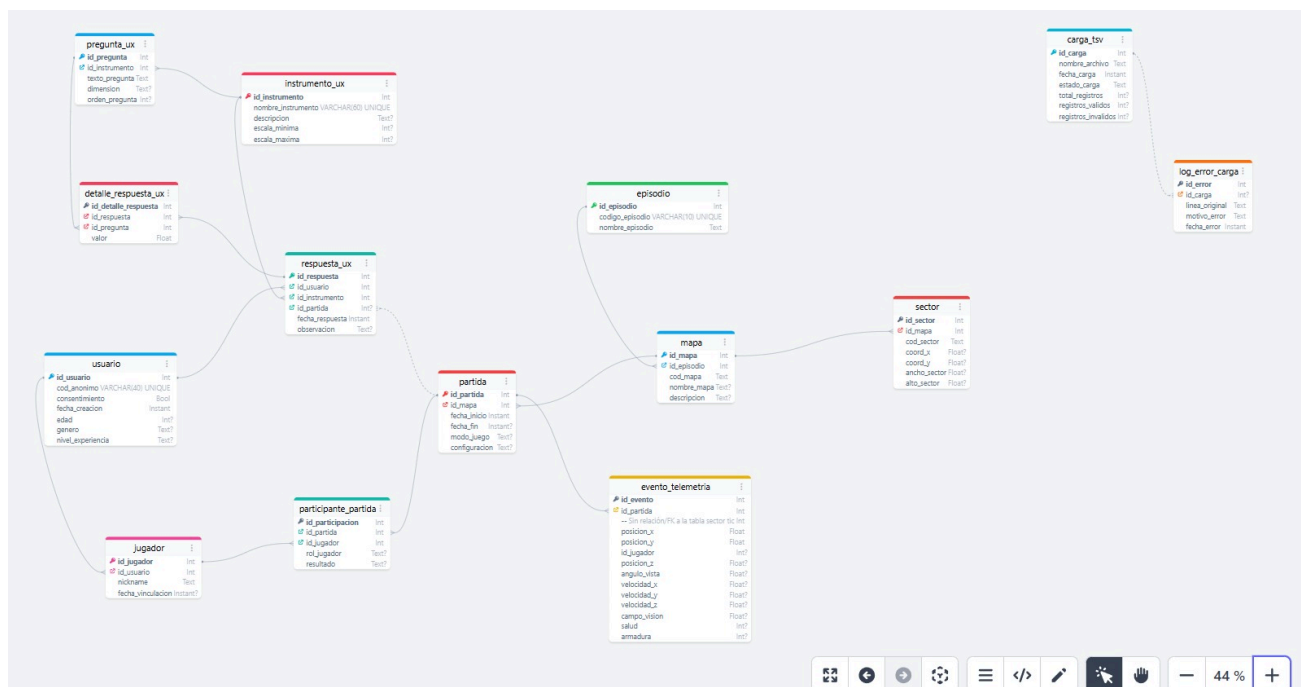
Estas tablas permiten almacenar diferentes encuestas (PENS, GUESS o BANGS), sus preguntas y las respuestas proporcionadas por los participantes.

Proceso ETL

Finalmente, se incluyeron las entidades “Carga_TSV” y “Log_Error_Carga”, encargadas de registrar la importación de archivos de telemetría y los posibles errores detectados durante el proceso ETL.

2.4 Diagrama Entidad-Relación

Una vez identificadas las entidades y relaciones, se construyó el diagrama entidad-relación del sistema. Este modelo representa la estructura conceptual completa de la base de datos y define cómo interactúan usuarios, jugadores, partidas, telemetría, mapas y encuestas UX.



Explicación: El modelo se estructura alrededor de las entidades Usuario, Jugador y Partida, que permiten identificar a los participantes y registrar su participación en las sesiones de juego. La entidad Participante_Partida gestiona la relación entre jugadores y partidas.

Para representar el entorno del videojuego, se incluyeron las entidades Episodio, Mapa y Sector, que organizan jerárquicamente los escenarios donde ocurren los eventos registrados.

La entidad Evento_Telemetría constituye el núcleo del sistema, ya que almacena información sobre posiciones, movimientos, velocidad, salud y demás variables capturadas durante el juego.

Además, se incorporó un módulo de experiencia de usuario compuesto por Instrumento_UX, Pregunta_UX, Respuesta_UX y Detalle_Respuesta_UX, permitiendo relacionar el comportamiento observado en las partidas con las respuestas de los participantes.

Finalmente, las entidades Carga_TSV y Log_Error_Carga permiten gestionar el proceso ETL, registrando las cargas de datos y los posibles errores detectados durante la importación.

En conjunto, el modelo proporciona una estructura organizada y normalizada que facilita el almacenamiento, consulta y análisis de los datos generados durante el proyecto.

2.5 Relaciones y Cardinalidades

El modelo incluye diferentes tipos de relaciones:

- Un usuario puede estar asociado a uno o varios jugadores.
- Un jugador puede participar en múltiples partidas.
- Una partida puede tener múltiples jugadores.
- Un episodio contiene varios mapas.
- Un mapa contiene múltiples sectores.
- Una partida genera múltiples eventos de telemetría.
- Un instrumento UX contiene múltiples preguntas.
- Una respuesta UX puede contener múltiples detalles de respuesta.

Estas cardinalidades garantizan una representación adecuada de la realidad del sistema y evitan redundancias en el almacenamiento de datos.

2.6 Diccionario de Datos

Después de validar el modelo conceptual, se construyó el diccionario de datos. Este documento especifica para cada tabla los atributos, tipos de datos, restricciones, claves primarias, claves foráneas y reglas de integridad necesarias para la implementación física de la base de

datos.(Ya explicados en el documento de la primera entrega)

Tabla	Campo	Tipo de dato	Restricciones	Descripción
usuario	id_usuario	INT	PK, NOT NULL	Identificador único del usuario voluntario.
usuario	cod_anonimo	VARCHAR(40)	UNIQUE, NOT NULL	Código anónimo usado para proteger la identidad del participante.
usuario	edad	INT	CHECK edad BETWEEN 10 AND 99	Edad del usuario voluntario.
usuario	genero	VARCHAR(30)	NULL	Género autodeclarado por el usuario.
usuario	nivel_experiencia	VARCHAR(30)	NULL	Nivel de experiencia previa con videojuegos.
usuario	consentimiento	BOOLEAN	NOT NULL, DEFAULT FALSE	Indica si el usuario aceptó participar en el estudio.
usuario	fecha_creacion	TIMESTAMP	NOT NULL	Fecha y hora en que se registró el usuario.
jugador	id_jugador	INT	PK, NOT NULL	Identificador único del jugador.
jugador	id_usuario	INT	FK → usuario(id_usuario), NOT NULL	Usuario real asociado al jugador.
jugador	nickname	VARCHAR(60)	NOT NULL	Alias o nombre usado dentro del juego.
jugador	fecha_vinculacion	TIMESTAMP	NULL	Fecha en que el alias fue asociado al usuario.
episodio	id_episodio	INT	PK, NOT NULL	Identificador único del episodio.

episodio	codigo_episodio	VARCHAR(10)	UNIQUE, NOT NULL	Código corto del episodio, por ejemplo E1.
episodio	nombre_episodio	VARCHAR(100)	NOT NULL	Nombre descriptivo del episodio.
mapa	id_mapa	INT	PK, NOT NULL	Identificador único del mapa.
mapa	id_episodio	INT	FK → episodio(id_episodio), NOT NULL	Episodio al que pertenece el mapa.
mapa	cod_mapa	VARCHAR(10)	NOT NULL	Código del mapa, por ejemplo E1M1.
mapa	nombre_mapa	VARCHAR(100)	NULL	Nombre descriptivo del mapa.
mapa	descripcion	TEXT	NULL	Información adicional del mapa.
sector	id_sector	INT	PK, NOT NULL	Identificador único del sector.
sector	id_mapa	INT	FK → mapa(id_mapa), NOT NULL	Mapa al que pertenece el sector.
sector	cod_sector	VARCHAR(30)	NOT NULL	Código del sector dentro del mapa.
sector	coord_x	NUMERIC(10,2)	NULL	Coordenada X de referencia del sector.
sector	coord_y	NUMERIC(10,2)	NULL	Coordenada Y de referencia del sector.
sector	ancho_sector	NUMERIC(10,2)	DEFAULT 250	Ancho aproximado del sector o celda.
sector	alto_sector	NUMERIC(10,2)	DEFAULT 250	Alto aproximado del sector o celda.
partida	id_partida	INT	PK, NOT NULL	Identificador único de la partida o sesión de juego.
partida	id_mapa	INT	FK → mapa(id_mapa), NOT NULL	Mapa donde se desarrolló la partida.
partida	fecha_inicio	TIMESTAMP	NOT NULL	Fecha y hora de inicio de la partida.

partida	fecha_fin	TIMESTAMP	CHECK fecha_fin >= fecha_inicio	Fecha y hora de finalización de la partida.
partida	modo_juego	VARCHAR(40)	NULL	Modo o tipo de partida jugada.
partida	configuracion	TEXT	NULL	Parámetros generales del motor o sesión.
participante_partida	id_participacion	INT	PK, NOT NULL	Identificador único de la participación.
participante_partida	id_partida	INT	FK → partida(id_partida), NOT NULL	Partida en la que participó el jugador.
participante_partida	id_jugador	INT	FK → jugador(id_jugador), NOT NULL	Jugador que participó en la partida.
participante_partida	rol_jugador	VARCHAR(40)	NULL	Rol del jugador dentro de la partida, si aplica.
participante_partida	resultado	VARCHAR(40)	NULL	Resultado general del jugador en la partida.
participante_partida	restriccion_unica	UNIQUE	UNIQUE(id_partida, id_jugador)	Evita registrar dos veces al mismo jugador en la misma partida.
evento telemetria	id_evento	BIGINT	PK, NOT NULL	Identificador único del evento de telemetría.
evento telemetria	id_partida	INT	FK → partida(id_partida), NOT NULL	Partida a la que pertenece el evento.
evento telemetria	id_jugador	INT	FK → jugador(id_jugador), NOT NULL	Jugador que generó el evento.
evento telemetria	id_sector	INT	FK → sector(id_sector), NULL	Sector donde se ubicaba el jugador.
evento telemetria	tiempo	INT	NOT NULL, CHECK tiempo >= 0	Unidad temporal del motor del juego.

evento_telemetria	posicion_x	NUMERIC(12,4)	NOT NULL	Posición del jugador en el eje X.
evento_telemetria	posicion_y	NUMERIC(12,4)	NOT NULL	Posición del jugador en el eje Y.
evento_telemetria	posicion_z	NUMERIC(12,4)	NULL	Posición del jugador en el eje Z.
evento_telemetria	angulo_vista	NUMERIC(8,4)	NULL	Ángulo de orientación del jugador o cámara.
evento_telemetria	velocidad_x	NUMERIC(10,4)	NULL	Componente X del movimiento o momentum.
evento_telemetria	velocidad_y	NUMERIC(10,4)	NULL	Componente Y del movimiento o momentum.
evento_telemetria	velocidad_z	NUMERIC(10,4)	NULL	Componente Z del movimiento o momentum.
evento_telemetria	campo_vision	NUMERIC(6,2)	NULL	Campo de visión o FOV del jugador.
evento_telemetria	salud	INT	CHECK salud >= 0	Nivel de vida del jugador.
evento_telemetria	armadura	INT	CHECK armadura >= 0	Nivel de armadura disponible.
evento_telemetria	municion	INT	CHECK municion >= 0	Cantidad de munición disponible.
evento_telemetria	restriccion_unica	UNIQUE	UNIQUE(id_partida, id_jugador, tic)	Evita duplicar eventos de telemetría para un mismo jugador en una misma partida.
instrumento_ux	id_instrumento	INT	PK, NOT NULL	Identificador único del instrumento UX.
instrumento_ux	nombre_instrumento	VARCHAR(60)	UNIQUE, NOT NULL	Nombre del instrumento, por

				ejemplo PENS, GUESS o BANGS.
instrumento_ux	descripcion	TEXT	NULL	Descripción general del instrumento.
instrumento_ux	escala_minima	INT	NULL	Valor mínimo permitido en la escala de respuesta.
instrumento_ux	escala_maxima	INT	NULL	Valor máximo permitido en la escala de respuesta.
pregunta_ux	id_pregunta	INT	PK, NOT NULL	Identificador único de la pregunta.
pregunta_ux	id_instrumento	INT	FK → instrumento_ux(id_instrumento), NOT NULL	Instrumento al que pertenece la pregunta.
pregunta_ux	texto_pregunta	TEXT	NOT NULL	Enunciado de la pregunta UX.
pregunta_ux	dimension	VARCHAR(60)	NULL	Dimensión que evalúa la pregunta, por ejemplo autonomía, competencia o satisfacción.
pregunta_ux	orden_pregunta	INT	NULL	Orden de aparición de la pregunta dentro del instrumento.
respuesta_ux	id_respuesta	INT	PK, NOT NULL	Identificador único de la encuesta respondida.
respuesta_ux	id_usuario	INT	FK → usuario(id_usuario), NOT NULL	Usuario que respondió el instrumento.
respuesta_ux	id_instrumento	INT	FK → instrumento_ux(id_instrumento), NOT NULL	Instrumento UX respondido.
respuesta_ux	id_partida	INT	FK → partida(id_partida), NULL	Partida asociada a la respuesta, si aplica.

respuesta_ux	fecha_respuesta	TIMESTAMP	NOT NULL	Fecha y hora en que se respondió la encuesta.
respuesta_ux	observacion	TEXT	NULL	Comentario adicional del usuario, si existe.
detalle_respuesta_ux	id_detalle_respuesta	INT	PK, NOT NULL	Identificador único del detalle de respuesta.
detalle_respuesta_ux	id_respuesta	INT	FK → respuesta_ux(id_respuesta), NOT NULL	Encuesta a la que pertenece la respuesta.
detalle_respuesta_ux	id_pregunta	INT	FK → pregunta_ux(id_pregunta), NOT NULL	Pregunta respondida.
detalle_respuesta_ux	valor	NUMERIC(5,2)	NOT NULL	Valor o puntuación asignada por el usuario.
detalle_respuesta_ux	restriccion_unica	UNIQUE	UNIQUE(id_respuesta, id_pregunta)	Evita registrar dos respuestas para la misma pregunta dentro de una encuesta.
carga_tsv	id_carga	INT	PK, NOT NULL	Identificador único de la carga realizada.
carga_tsv	nombre_archivo	VARCHAR(150)	NOT NULL	Nombre del archivo TSV cargado.
carga_tsv	fecha_carga	TIMESTAMP	NOT NULL	Fecha y hora de la carga del archivo.
carga_tsv	estado_carga	VARCHAR(30)	NOT NULL	Estado de la carga: pendiente, validada o rechazada.
carga_tsv	total_registros	INT	DEFAULT 0	Cantidad total de registros procesados.
carga_tsv	registros_validos	INT	DEFAULT 0	Cantidad de registros aceptados.
carga_tsv	registros_invalidos	INT	DEFAULT 0	Cantidad de registros rechazados.

log_error_carga	id_error	INT	PK, NOT NULL	Identificador único del error registrado.
log_error_carga	id_carga	INT	FK → carga_tsv(id_carga), NULL	Carga a la que pertenece el error.
log_error_carga	linea_original	TEXT	NOT NULL	Línea original del archivo que generó el error.
log_error_carga	motivo_error	VARCHAR(200)	NOT NULL	Motivo del rechazo del registro.
log_error_carga	fecha_error	TIMESTAMP	NOT NULL, DEFAULT NOW()	Fecha y hora en que se registró el error.

2.7 Resultado de la Primera Entrega

Como resultado de esta primera fase se obtuvo un modelo relacional completo, normalizado y preparado para soportar el almacenamiento de grandes volúmenes de datos de telemetría y encuestas UX. La definición adecuada de entidades, relaciones y restricciones permitió establecer una base sólida para las etapas posteriores de implementación, carga de datos y análisis.

Esta versión se ve mucho más profesional porque explica cada bloque del diagrama ER corregido y justifica por qué existe cada entidad dentro del proyecto.

3. Segunda Entrega: Implementación y ETL

3.1 Objetivo

El objetivo de esta etapa fue transformar el modelo conceptual y lógico diseñado en la primera entrega en una base de datos completamente funcional. Para ello se implementaron todas

las tablas, restricciones y relaciones definidas en el diagrama ER, además de desarrollar un proceso ETL que permitiera importar y transformar los datos de telemetría generados por el videojuego DOOM.

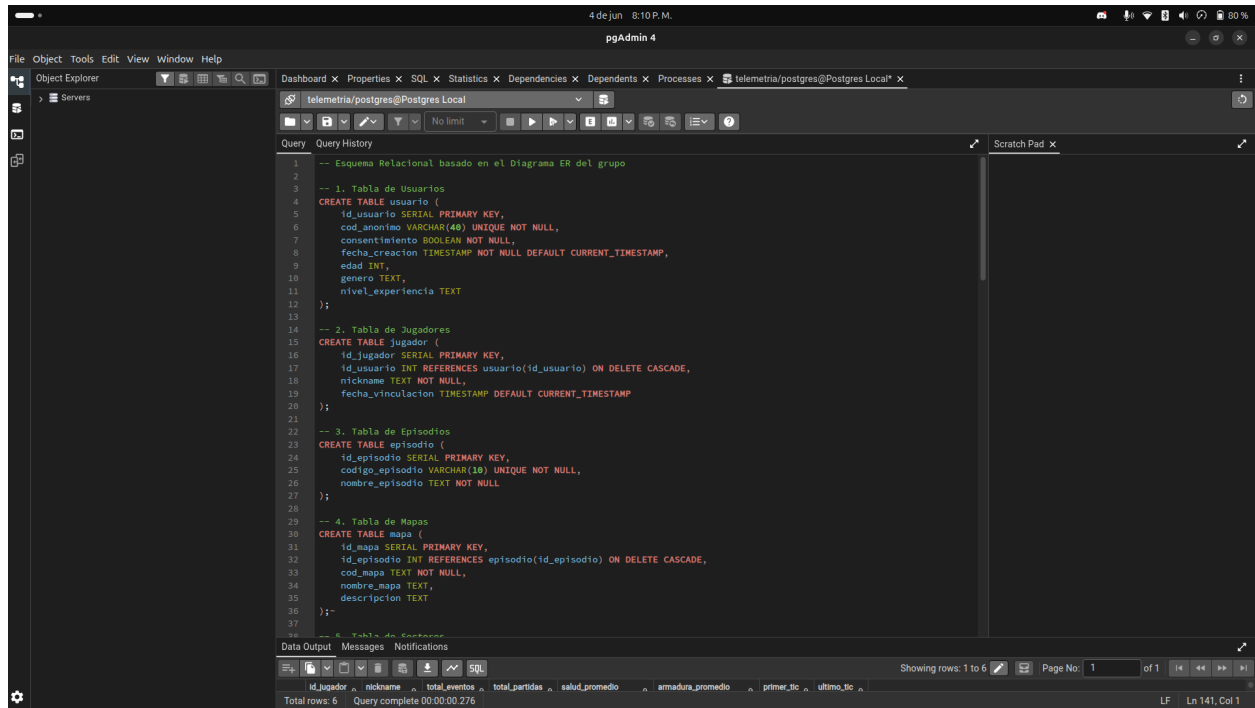
Esta fase representó la transición desde el diseño teórico hacia una solución operativa capaz de almacenar grandes volúmenes de información y garantizar la integridad de los datos.

3.2 Implementación de la Base de Datos

A partir del modelo relacional diseñado previamente, se crearon todas las tablas necesarias dentro de PostgreSQL. Durante esta implementación se definieron claves primarias, claves foráneas, restricciones de unicidad y validaciones para garantizar la consistencia de la información almacenada.

Las tablas implementadas permitieron representar todos los elementos identificados durante la fase de diseño, incluyendo usuarios, jugadores, partidas, mapas, sectores, eventos de telemetría, instrumentos UX y respuestas de encuestas.

La correcta implementación de estas estructuras permitió asegurar la integridad referencial entre las diferentes entidades del sistema.

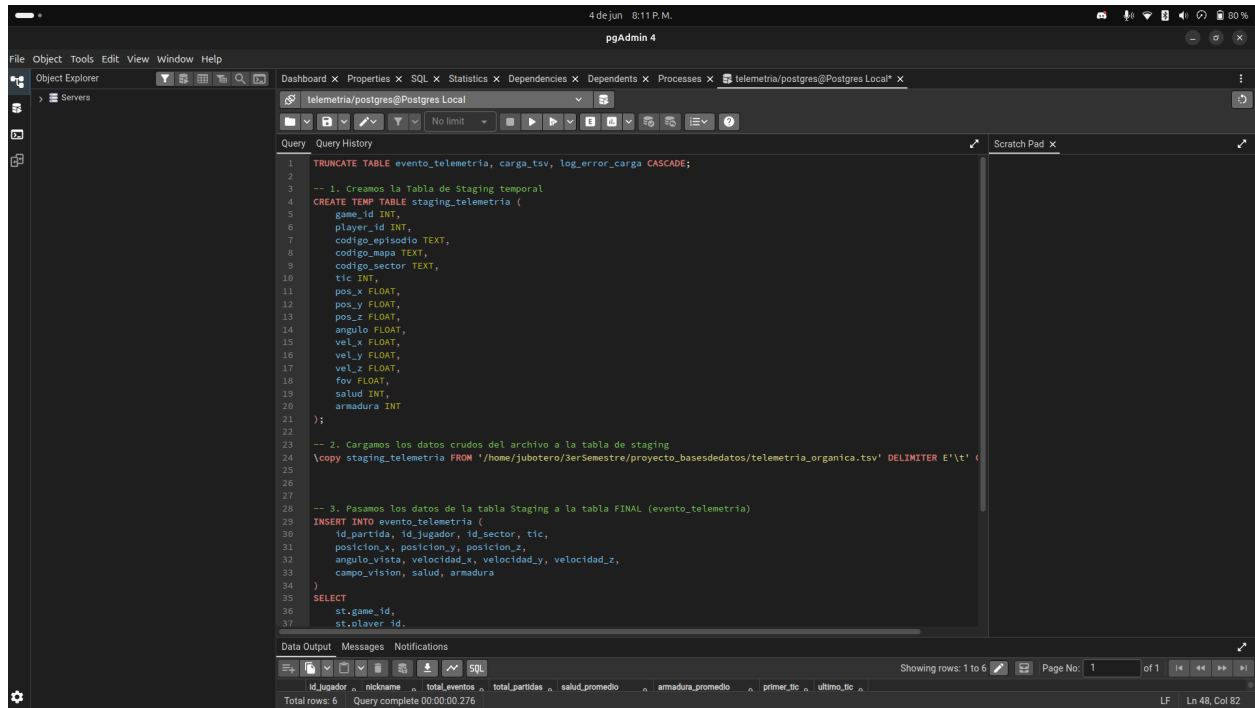


3.3 Proceso ETL

Uno de los componentes más importantes de esta entrega fue el desarrollo del proceso ETL (Extract, Transform, Load), encargado de integrar los datos generados por el videojuego con la base de datos relacional.

Durante la etapa de extracción se obtuvieron archivos de telemetría en formato TSV generados a partir de partidas instrumentadas de DOOM. Posteriormente, estos datos fueron cargados inicialmente en tablas de staging, donde se realizaron procesos de validación y limpieza. Finalmente, la información transformada fue insertada en las tablas principales del sistema.

Este enfoque permitió separar los datos originales de los datos ya validados, facilitando el control de calidad y reduciendo errores durante la carga.



3.4 Carga de Datos

Como parte del proceso ETL se implementaron tablas temporales de staging. Estas tablas actuaron como una zona intermedia donde se recibieron los datos provenientes de los archivos TSV antes de ser incorporados a las tablas definitivas.

El uso de staging permitió:

- Detectar registros incompletos o/u inválidos.
- Verificar formatos de fechas y valores numéricos.
- Validar referencias hacia partidas, jugadores y sectores.
- Mantener trazabilidad sobre el origen de los datos.

Además, se incorporaron mecanismos de control mediante las tablas de carga y registro de errores.


```
Query  Query History
1  INSERT INTO evento_telemetria (
2      id_partida, id_jugador, id_sector, tic,
3      posicion_x, posicion_y, posicion_z,
4      angulo_vista, velocidad_x, velocidad_y, velocidad_z,
5      campo_vision, salud, armadura
6  )
7  SELECT
8      st.game_id,
9      st.player_id,
10     s.id_sector, -- <-- TRANSFORMACIÓN: Cambiamos texto por el ID numérico
11     st.tic,
12     st.pos_x, st.pos_y, st.pos_z,
13     st.angulo,
14     st.vel_x, st.vel_y, st.vel_z,
15     st.fov,
16     st.salud, st.armadura
17 FROM staging_telemetria st
18 -- Las validaciones (Cruzamos datos de texto para hallar su llave foránea real)
19 JOIN mapa m ON m.cod_mapa = TRIM(st.codigo_mapa)
20 JOIN sector s ON s.cod_sector = TRIM(st.codigo_sector) AND s.id_mapa = m.id_mapa;
```

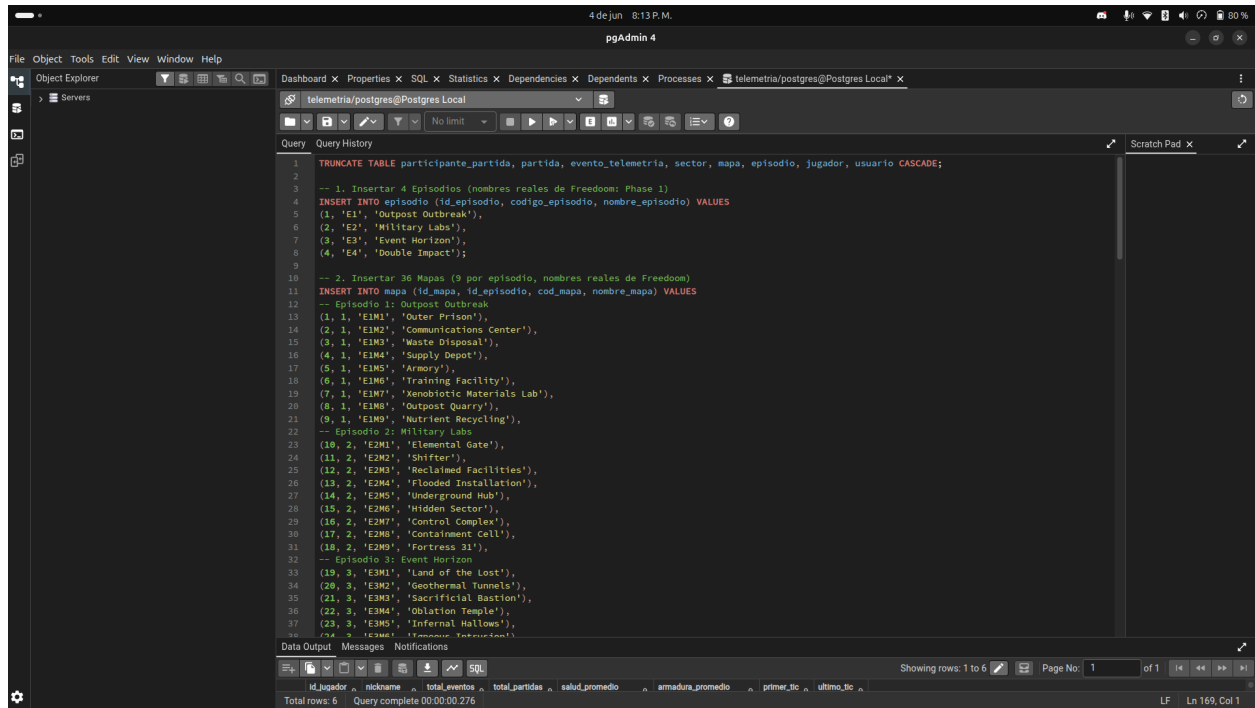
3.5 Carga de Datos Maestros

Antes de importar la telemetría fue necesario poblar las tablas maestras del sistema. Estas tablas contienen información relativamente estática que sirve de referencia para otras entidades.

Entre los datos cargados se encuentran:

- Usuarios participantes.
- Jugadores asociados.
- Episodios del juego.
- Mapas.
- Sectores.
- Instrumentos UX.
- Preguntas de las encuestas.

La carga previa de esta información permitió mantener la integridad referencial durante el proceso ETL.



3.6 Importación de Telemetría

Una vez preparada la estructura de la base de datos, se procedió a importar los archivos de telemetría generados durante las partidas.

Cada registro de telemetría contiene información relacionada con:

- Posición del jugador.
- Velocidad de desplazamiento.
- Dirección de visión.
- Salud.
- Armadura.
- Instante temporal dentro de la partida.

Estos registros constituyen el conjunto de datos principal utilizado posteriormente para realizar análisis y consultas analíticas.

game_id	player_id	codigo_episodio	codigo_mapa	codigo_sector	tic	pos_x	pos_y	pos_z	angulo	vel_x	vel_y	vel_z	fov	salud	armadura
1	1	E1	E1M6	SEC 01 188 0	-160	0	98.00	-1 1	0	90.0	100 0				
2	1	E1	E1M6	SEC 01 189 -1	-189	0	98.00	-1 2	0	90.0	100 0				
3	1	E1	E1M6	SEC 01 190 -1	-156	0	98.00	-1 4	0	90.0	100 0				
4	1	E1	E1M6	SEC 01 191 -1	-152	0	98.00	-1 5	0	90.0	100 0				
5	1	E1	E1M6	SEC 01 192 -1	-146	0	98.00	-2 6	0	90.0	100 0				
6	1	E1	E1M6	SEC 01 193 -2	-140	0	98.00	-3 7	0	90.0	100 0				
7	1	E1	E1M6	SEC 01 194 -4	-132	0	98.00	-4 8	0	90.0	100 0				
8	1	E1	E1M6	SEC 01 195 -8	-124	0	98.00	-5 9	0	90.0	100 0				
9	1	E1	E1M1	SEC 01 197 -416	256	0	359.82	0 -1	0	90.0	100 0				
10	1	E1	E1M1	SEC 01 198 -416	255	0	359.82	0 -1	0	90.0	100 0				
11	1	E1	E1M1	SEC 01 212 -416	255	0	359.87	0 -1	0	90.0	100 0				
12	1	E1	E1M1	SEC 01 213 -416	255	0	359.43	0 -1	0	90.0	100 0				
13	1	E1	E1M1	SEC 01 214 -416	255	0	359.30	0 -1	0	90.0	100 0				
14	1	E1	E1M1	SEC 01 215 -415	255	0	359.30	0 -1	0	90.0	100 0				
15	1	E1	E1M1	SEC 01 216 -414	255	0	359.30	0 -1	0	90.0	100 0				
16	1	E1	E1M1	SEC 01 217 -413	255	0	359.30	0 -1	0	90.0	100 0				
17	1	E1	E1M1	SEC 01 218 -413	255	0	359.30	0 -1	0	90.0	100 0				
18	1	E1	E1M1	SEC 01 219 -412	255	0	359.30	0 -1	0	90.0	100 0				
19	1	E1	E1M1	SEC 01 220 -411	255	0	359.30	0 -1	0	90.0	100 0				
20	1	E1	E1M1	SEC 01 221 -411	255	0	359.30	0 -1	0	90.0	100 0				
21	1	E1	E1M1	SEC 01 222 -411	255	0	359.30	0 -1	0	90.0	100 0				
22	1	E1	E1M1	SEC 01 223 -410	255	0	359.30	0 -1	0	90.0	100 0				
23	1	E1	E1M1	SEC 01 224 -410	255	0	359.30	0 -1	0	90.0	100 0				
24	1	E1	E1M1	SEC 01 225 -409	255	0	4.22	0 -1	0	90.0	100 0				
25	1	E1	E1M1	SEC 01 226 -409	255	0	16.70	-1 -1	0	90.0	100 0				
26	1	E1	E1M1	SEC 01 227 -410	255	0	31.03	-1 -1	0	90.0	100 0				
27	1	E1	E1M1	SEC 01 228 -411	255	0	43.86	-2 -1	0	90.0	100 0				
28	1	E1	E1M1	SEC 01 229 -412	254	0	61.52	-2 -3	0	90.0	100 0				
29	1	E1	E1M1	SEC 01 230 -414	251	0	87.01	-2 -3	0	90.0	100 0				
30	1	E1	E1M1	SEC 01 231 -416	249	0	112.15	-2 -3	0	90.0	100 0				
31	1	E1	E1M1	SEC 01 232 -418	246	0	127.09	-3 -2	0	90.0	100 0				
32	1	E1	E1M1	SEC 01 233 -420	245	0	128.85	-2 -2	0	90.0	100 0				
33	1	E1	E1M1	SEC 01 234 -422	244	0	128.85	-2 -2	0	90.0	100 0				
34	1	E1	E1M1	SEC 01 235 -423	243	0	128.85	-2 -2	0	90.0	100 0				
35	1	E1	E1M1	SEC 01 236 -425	242	0	128.85	-2 -1	0	90.0	100 0				
36	1	E1	E1M1	SEC 01 237 -426	241	0	128.85	-2 -1	0	90.0	100 0				
37	1	E1	E1M1	SEC 01 238 -428	240	0	128.85	-2 -1	0	90.0	100 0				
38	1	E1	E1M1	SEC 01 239 -429	239	0	128.85	-2 -1	0	90.0	100 0				
39	1	E1	E1M1	SEC 01 240 -430	238	0	129.02	-2 -1	0	90.0	100 0				
40	1	E1	E1M1	SEC 01 241 -431	238	0	129.64	-2 0	0	90.0	100 0				
41	1	E1	E1M1	SEC 01 242 -433	238	0	129.62	-2 0	0	90.0	100 0				
42	1	E1	E1M1	SEC 01 243 -435	239	0	127.71	-3 1	0	90.0	100 0				
43	1	E1	E1M1	SEC 01 244 -437	240	0	127.18	-3 2	0	90.0	100 0				
44	1	E1	E1M1	SEC 01 245 -439	242	0	127.79	-4 3	0	90.0	100 0				
45	1	E1	E1M1	SEC 01 246 -443	246	0	129.73	-5 4	0	90.0	100 0				
46	1	E1	E1M1	SEC 01 247 -447	250	0	133.23	-5 4	0	90.0	100 0				
47	1	E1	E1M1	SEC 01 248 -452	255	0	138.25	-6 5	0	90.0	100 0				
48	1	E1	E1M1	SEC 01 249 -457	260	0	143.53	-7 5	0	90.0	100 0				

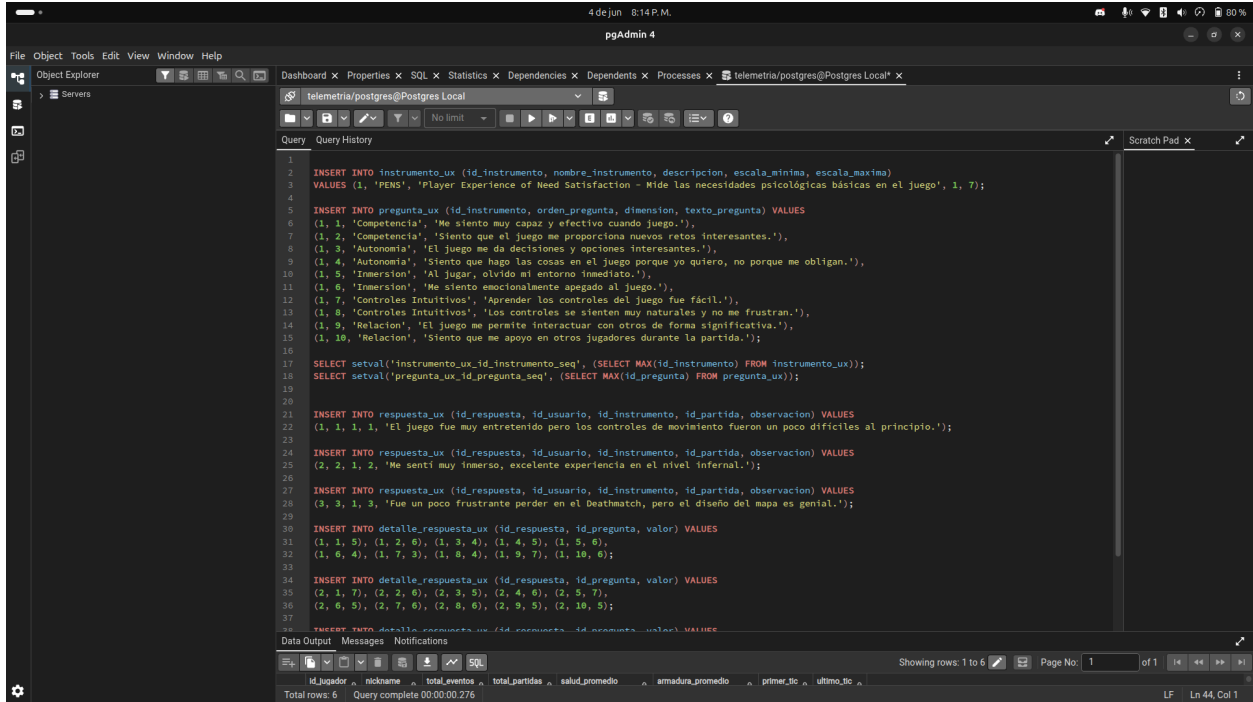
3.7 Carga de Encuestas UX

Además de la telemetría, se incorporaron las respuestas obtenidas mediante los instrumentos UX seleccionados para el proyecto.

Las respuestas fueron almacenadas utilizando la estructura definida por las entidades:

- Instrumento_UX
- Pregunta_UX
- Respuesta_UX
- Detalle_Respuesta_UX

Gracias a esta estructura fue posible relacionar posteriormente los comportamientos observados en las partidas con la percepción subjetiva de los usuarios.



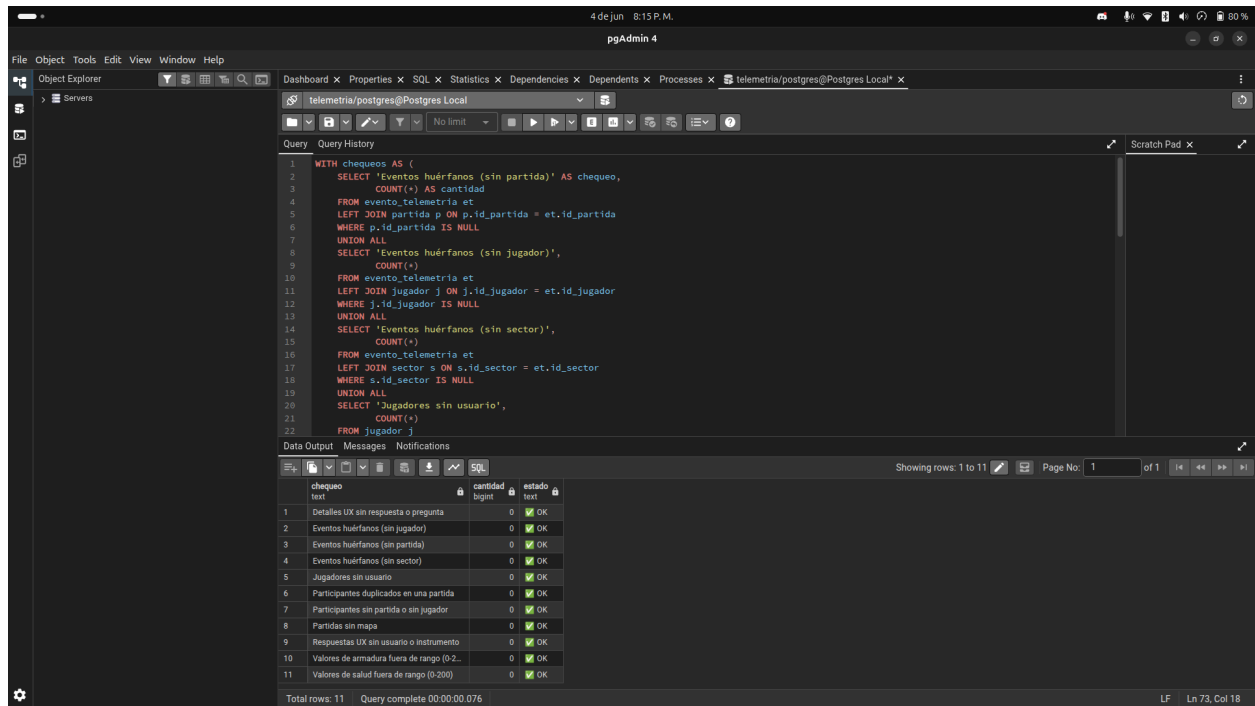
3.8 Validación de Integridad

Una vez finalizada la carga de datos, se realizaron pruebas para verificar que todas las relaciones funcionaran correctamente y que no existieran registros huérfanos o inconsistencias.

Se validó especialmente:

- Correspondencia entre usuarios y jugadores.
- Participación de jugadores en partidas.
- Relación entre partidas y eventos de telemetría.
- Asociación entre encuestas y participantes.

Estas pruebas garantizan que la base de datos estuviera lista para soportar las consultas analíticas de la siguiente fase.



3.9 Resultado de la Segunda Entrega

Como resultado de esta etapa se obtuvo una base de datos completamente implementada y operativa en PostgreSQL, capaz de almacenar información de usuarios, partidas, telemetría y experiencia de usuario. Además, se desarrolló un proceso ETL funcional que permite importar datos desde archivos externos de manera controlada y reproducible.

La culminación de esta fase proporcionó la infraestructura necesaria para la tercera entrega, enfocada en el análisis de datos, optimización de consultas e interpretación de resultados.

4. Tercera Entrega: Consultas y Optimización

4.1 Objetivo

El objetivo de esta fase fue explotar la información almacenada en la base de datos mediante consultas analíticas que permitieran estudiar el comportamiento de los jugadores

durante las partidas de DOOM. Adicionalmente, se implementaron mecanismos de optimización para mejorar el rendimiento de las consultas y facilitar el acceso a la información más relevante.

Esta etapa representa la aplicación práctica de la base de datos desarrollada en las entregas anteriores, transformando los datos almacenados en información útil para el análisis del comportamiento de los usuarios y la experiencia de juego.

4.2 Consultas Analíticas

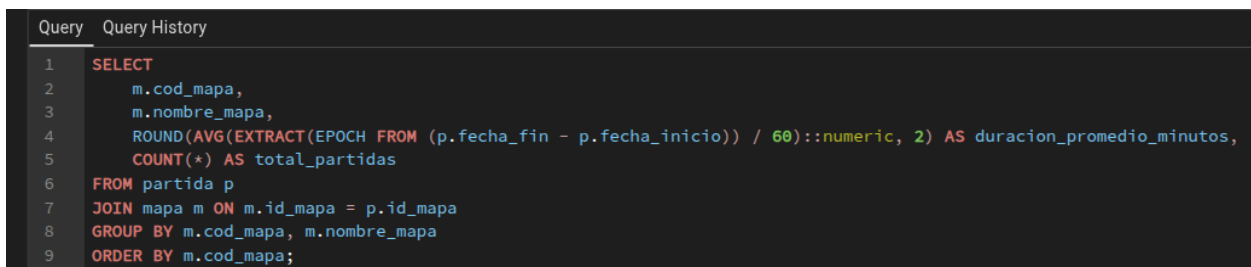
A partir de la telemetría recopilada y de las respuestas de las encuestas UX, se implementaron diversas consultas analíticas orientadas a responder preguntas relacionadas con el comportamiento de los jugadores dentro del entorno del videojuego.

Estas consultas permitieron identificar patrones de movimiento, interacción entre jugadores, permanencia en determinadas zonas del mapa y posibles relaciones entre el comportamiento observado y la percepción de la experiencia de usuario.

Consulta Analítica 1

Objetivo: Duración promedio de sesiones por mapa

Consulta SQL:

A screenshot of a SQL query editor with a dark background. The query is displayed in a monospaced font with syntax highlighting. It is a SELECT statement that calculates the average duration of game sessions per map. The query includes columns for map code, map name, average duration in minutes (rounded to 2 decimal places), and the total number of games. It uses a JOIN between the 'partida' table and the 'mapa' table, and is grouped by map code and name, ordered by map code.

```
Query  Query History
1  SELECT
2      m.cod_mapa,
3      m.nombre_mapa,
4      ROUND(AVG(EXTRACT(EPOCH FROM (p.fecha_fin - p.fecha_inicio)) / 60)::numeric, 2) AS duracion_promedio_minutos,
5      COUNT(*) AS total_partidas
6  FROM partida p
7  JOIN mapa m ON m.id_mapa = p.id_mapa
8  GROUP BY m.cod_mapa, m.nombre_mapa
9  ORDER BY m.cod_mapa;
```

Resultado obtenido:

Data Output Messages Notifications				
	cod_mapa text	nombre_mapa text	duracion_promedio_minutos numeric	total_partidas bigint
1	E1M1	Outer Prison	30.00	1
2	E2M1	Elemental Gate	45.00	1
3	E3M1	Land of the Lost	20.00	1

Consulta Analítica 2

Objetivo: Jugadores con mayor proximidad promedio

Consulta SQL:

```

Query Query History
1  SELECT
2    j1.nickname AS jugador1,
3    j2.nickname AS jugador2,
4    ROUND(AVG(SQRT(POW(e1.posicion_x - e2.posicion_x, 2) + POW(e1.posicion_y - e2.posicion_y, 2))))::numeric, 2) AS distancia_promedio,
5    COUNT(*) AS tics_compartidos
6  FROM evento_telemetria e1
7  JOIN evento_telemetria e2
8    ON e1.id_partida = e2.id_partida
9    AND e1.tic = e2.tic
10   AND e1.id_jugador < e2.id_jugador
11  JOIN jugador j1 ON j1.id_jugador = e1.id_jugador
12  JOIN jugador j2 ON j2.id_jugador = e2.id_jugador
13  GROUP BY j1.nickname, j2.nickname
14  ORDER BY distancia_promedio;

```

Resultado obtenido:

Data Output Messages Notifications				
	jugador1 text	jugador2 text	distancia_promedio numeric	tics_compartidos bigint
1	DoomSlayer...	DoomSlayer...	808.06	4897
2	DoomSlayer...	DoomSlayer...	1238.14	18418
3	DoomSlayer...	DoomSlayer...	2895.48	10822

Consulta Analítica 3

Objetivo: Distancias de trayectoria mín/máx por jugador

Consulta SQL:

```
Query Query History
1 WITH distancias_por_tic AS (
2     SELECT
3         id_jugador,
4         id_partida,
5         tic,
6         posicion_x,
7         posicion_y,
8         posicion_z,
9         LAG(posicion_x) OVER (PARTITION BY id_jugador, id_partida ORDER BY tic) AS prev_x,
10        LAG(posicion_y) OVER (PARTITION BY id_jugador, id_partida ORDER BY tic) AS prev_y,
11        LAG(posicion_z) OVER (PARTITION BY id_jugador, id_partida ORDER BY tic) AS prev_z
12    FROM evento_telemetria
13 ),
14 trayectoria_por_partida AS (
15     SELECT
16         id_jugador,
17         id_partida,
18         ROUND(SUM(SQRT(POW(posicion_x - prev_x, 2) + POW(posicion_y - prev_y, 2) + POW(posicion_z - prev_z, 2))):numeric, 2) AS distancia_total
19    FROM distancias_por_tic
20   WHERE prev_x IS NOT NULL
21   GROUP BY id_jugador, id_partida
22 )
23 SELECT
24     j.nickname,
25     ROUND(MIN(tp.distancia_total)::numeric, 2) AS distancia_minima,
26     ROUND(MAX(tp.distancia_total)::numeric, 2) AS distancia_maxima,
27     ROUND(AVG(tp.distancia_total)::numeric, 2) AS distancia_promedio
28 FROM trayectoria_por_partida tp
29 JOIN jugador j ON j.id_jugador = tp.id_jugador
30 GROUP BY j.id_jugador, j.nickname
31 ORDER BY j.nickname;
```

Resultado obtenido:

Data Output Messages Notifications				
SQL				
	nickname text	distancia_minima numeric	distancia_maxima numeric	distancia_promedio numeric
1	DoomSlayer...	138761.54	138761.54	138761.54
2	DoomSlayer...	139709.12	139709.12	139709.12
3	DoomSlayer...	164419.23	164419.23	164419.23
4	DoomSlayer...	122442.77	122442.77	122442.77
5	DoomSlayer...	182316.81	182316.81	182316.81
6	DoomSlayer...	41890.98	41890.98	41890.98

Consulta Analítica 4

Objetivo: Respuestas UX para jugadores con trayectorias de duración superior al promedio

Consulta SQL:

QueryQuery History

```
1  WITH duracion_jugador AS (  
2      SELECT  
3          id_jugador,  
4          id_partida,  
5          MAX(tic) - MIN(tic) + 1 AS duracion_tics  
6      FROM evento_telemetria  
7      GROUP BY id_jugador, id_partida  
8  ),  
9  promedio_global AS (  
10     SELECT AVG(duracion_tics) AS promedio_global_tics FROM duracion_jugador  
11 ),  
12 jugadores_sobre_promedio AS (  
13     SELECT DISTINCT dj.id_jugador, dj.id_partida  
14     FROM duracion_jugador dj, promedio_global pg  
15     WHERE dj.duracion_tics > pg.promedio_global_tics  
16 )  
17 SELECT  
18     j.nickname,  
19     ru.id_respuesta,  
20     ru.observacion,  
21     ROUND(AVG(dr.valor)::numeric, 2) AS puntaje_ux_promedio  
22 FROM jugadores_sobre_promedio jsp  
23 JOIN jugador j ON j.id_jugador = jsp.id_jugador  
24 JOIN respuesta_ux ru ON ru.id_usuario = j.id_usuario AND ru.id_partida = jsp.id_partida  
25 JOIN detalle_respuesta_ux dr ON dr.id_respuesta = ru.id_respuesta  
26 GROUP BY j.nickname, ru.id_respuesta, ru.observacion  
27 ORDER BY puntaje_ux_promedio DESC;
```

Resultado obtenido:

Data OutputMessagesNotifications				
	nickname text	id_respuesta integer	observacion text	puntaje_ux_promedio numeric
1	DoomSlayer...	2	Me sentí muy inmerso, excelente experiencia en el nivel infernal.	5.80
2	DoomSlayer...	3	Fue un poco frustrante perder en el Deathmatch, pero el diseño del mapa es genial.	5.30
3	DoomSlayer...	1	El juego fue muy entretenido pero los controles de movimiento fueron un poco difíciles al princ...	5.00

Consulta Analítica 5

Objetivo: Sector más visitado (punto caliente) por episodio/mapa

Consulta SQL:

Query	Query History
1	SELECT
2	e1.id_partida,
3	s.cod_sector,
4	m.cod_mapa,
5	j1.nickname AS jugador1,
6	j2.nickname AS jugador2,
7	COUNT (DISTINCT e1.tic) AS tics_copresencia
8	FROM evento_telemetria e1
9	JOIN evento_telemetria e2
10	ON e1.id_partida = e2.id_partida
11	AND e1.tic = e2.tic
12	AND e1.id_sector = e2.id_sector
13	AND e1.id_jugador < e2.id_jugador
14	JOIN sector s ON s.id_sector = e1.id_sector
15	JOIN mapa m ON m.id_mapa = s.id_mapa
16	JOIN jugador j1 ON j1.id_jugador = e1.id_jugador
17	JOIN jugador j2 ON j2.id_jugador = e2.id_jugador
18	GROUP BY e1.id_partida, s.cod_sector, m.cod_mapa, j1.nickname, j2.nickname
19	ORDER BY tics_copresencia DESC ;

Resultado obtenido:

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	id_partida integer	cod_sector text	cod_mapa text	jugador1 text	jugador2 text	tics_copresencia bigint
1	1	SEC_02	E1M2	DoomSlayer...	DoomSlayer...	9271
2	1	SEC_02	E1M1	DoomSlayer...	DoomSlayer...	5392
3	3	SEC_01	E3M1	DoomSlayer...	DoomSlayer...	4381
4	2	SEC_01	E2M1	DoomSlayer...	DoomSlayer...	1951
5	2	SEC_02	E2M1	DoomSlayer...	DoomSlayer...	485
6	1	SEC_01	E1M1	DoomSlayer...	DoomSlayer...	240
7	3	SEC_01	E1M6	DoomSlayer...	DoomSlayer...	96
8	2	SEC_01	E1M6	DoomSlayer...	DoomSlayer...	79
9	3	SEC_02	E3M1	DoomSlayer...	DoomSlayer...	32

4.3 Índices

Con el fin de mejorar el rendimiento de las consultas analíticas, se implementaron índices sobre los atributos más utilizados en filtros, agrupamientos y relaciones entre tablas.

Los índices permiten reducir el tiempo necesario para localizar registros dentro de tablas con grandes volúmenes de datos, especialmente en la entidad de eventos de telemetría, que concentra la mayor cantidad de registros del sistema.

Imagen: Índices Implementados

Query	Query History
1	<code>CREATE INDEX idx_telemetria_tiempo</code>
2	<code>ON evento_telemetria (id_partida, id_jugador, tic);</code>
3	

Query	Query History
1	<code>CREATE INDEX idx_telemetria_sector</code>
2	<code>ON evento_telemetria (id_partida, id_sector);</code>

Query	Query History
1	<code>CREATE INDEX idx_telemetria_espacial</code>
2	<code>ON evento_telemetria USING gist (point(posicion_x, posicion_y));</code>

Query	Query History
1	<code>CREATE INDEX idx_participante_jp</code>
2	<code>ON participante_partida (id_jugador, id_partida);</code>

4.4 Comparación de Rendimiento

	SIN INDICE	CON INDICE
CONSULTA 1	10.076 ms	9.493 ms
CONSULTA 2	21.657 ms	16.210 ms
CONSULTA 3	11.069 ms	0.855 ms
CONSULTA 4	0.336 ms	0.064 ms

Para medir el impacto de los índices se utilizó la herramienta **EXPLAIN ANALYZE** de PostgreSQL. Esta herramienta permitió observar el plan de ejecución de las consultas y comparar el tiempo requerido antes y después de la optimización.

Los resultados obtenidos evidenciaron mejoras significativas en el rendimiento de las consultas más complejas, especialmente aquellas que procesan grandes cantidades de eventos de telemetría.

4.4.1 Consulta de trayectoria por jugador

Antes:

	QUERY PLAN text
1	Sort (cost=2847.84..2869.16 rows=8531 width=32) (actual time=8.043..8.944 rows=23282.00 loops=1)
2	Sort Key: tic
3	Sort Method: quicksort Memory: 2042kB
4	Buffers: shared hit=455
5	-> Bitmap Heap Scan on evento_telemetria (cost=223.86..2290.83 rows=8531 width=32) (actual time=0.788..3.794 rows=23282.00 loops=1)
6	Recheck Cond: ((id_partida = 1) AND (id_jugador = 1))
7	Heap Blocks: exact=363
8	Buffers: shared hit=455
9	-> Bitmap Index Scan on idx_telemetria_tiempo (cost=0.00..221.73 rows=8531 width=0) (actual time=0.737..0.737 rows=23282.00 loops=1)
10	Index Cond: ((id_partida = 1) AND (id_jugador = 1))
11	Index Searches: 1
12	Buffers: shared hit=92
13	Planning Time: 0.125 ms
14	Execution Time: 10.076 ms



Después:

	QUERY PLAN	
	text	
1	Sort (cost=2847.84..2869.16 rows=8531 width=32) (actual time=7.740..8.642 rows=23282.00 loops=1)	
2	Sort Key: tic	
3	Sort Method: quicksort Memory: 2042kB	
4	Buffers: shared hit=363 read=92	
5	-> Bitmap Heap Scan on evento telemetria (cost=223.86..2290.83 rows=8531 width=32) (actual time=1.367..4.309 rows=23282.00 loops=1)	
6	Recheck Cond: ((id_partida = 1) AND (id_jugador = 1))	
7	Heap Blocks: exact=363	
8	Buffers: shared hit=363 read=92	
9	-> Bitmap Index Scan on idx telemetria tiempo (cost=0.00..221.73 rows=8531 width=0) (actual time=1.318..1.318 rows=23282.00 loops=1)	
10	Index Cond: ((id_partida = 1) AND (id_jugador = 1))	
11	Index Searches: 1	
12	Buffers: shared read=92	
13	Planning:	
14	Buffers: shared hit=22 read=1	
15	Planning Time: 0.237 ms	
16	Execution Time: 9.493 ms	

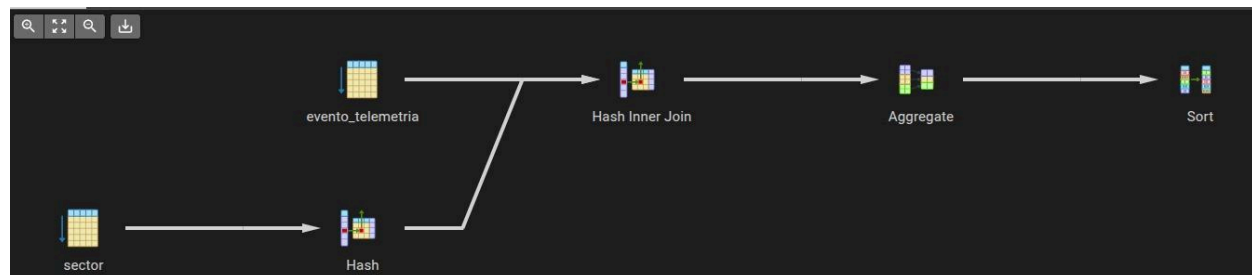


En esta consulta se evaluó el rendimiento al consultar la trayectoria de un jugador específico dentro de una partida, ordenando los datos por tic.

4.4.2 Consulta de sectores más visitados

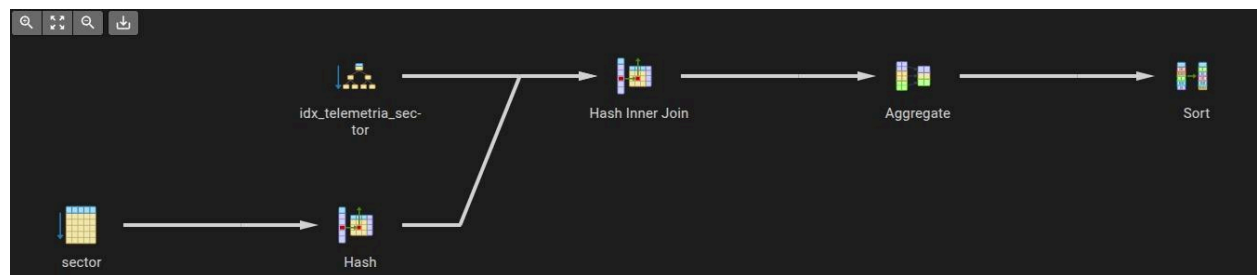
Antes:

	QUERY PLAN	
	text	🔒
1	Sort (cost=3991.14..3991.23 rows=34 width=23) (actual time=21.614..21.617 rows=6.00 loops=1)	
2	Sort Key: (count(*)) DESC	
3	Sort Method: quicksort Memory: 25kB	
4	Buffers: shared hit=1940	
5	-> HashAggregate (cost=3989.94..3990.28 rows=34 width=23) (actual time=21.605..21.609 rows=6.00 loops=1)	
6	Group Key: t.id_sector, s.cod_sector	
7	Batches: 1 Memory Usage: 32kB	
8	Buffers: shared hit=1940	
9	-> Hash Join (cost=2.62..3644.53 rows=46054 width=15) (actual time=1.088..14.805 rows=45511.00 loops=1)	
10	Hash Cond: (t.id_sector = s.id_sector)	
11	Buffers: shared hit=1940	
12	-> Seq Scan on evento_telemetria t (cost=0.00..3513.90 rows=46054 width=8) (actual time=0.012..7.334 rows=45511.00 loo...	
13	Filter: (id_partida = 1)	
14	Rows Removed by Filter: 80481	
15	Buffers: shared hit=1939	
16	-> Hash (cost=1.72..1.72 rows=72 width=11) (actual time=0.023..0.024 rows=72.00 loops=1)	
17	Buckets: 1024 Batches: 1 Memory Usage: 12kB	
18	Buffers: shared hit=1	
19	-> Seq Scan on sector s (cost=0.00..1.72 rows=72 width=11) (actual time=0.008..0.013 rows=72.00 loops=1)	
20	Buffers: shared hit=1	
21	Planning:	
22	Buffers: shared hit=5	
23	Planning Time: 0.257 ms	
24	Execution Time: 21.657 ms	



Después:

	QUERY PLAN	
	text	
1	Sort (cost=1443.48..1443.56 rows=34 width=23) (actual time=16.161..16.164 rows=6.00 loops=1)	
2	Sort Key: (count(*)) DESC	
3	Sort Method: quicksort Memory: 25kB	
4	Buffers: shared hit=2 read=40	
5	-> HashAggregate (cost=1442.27..1442.61 rows=34 width=23) (actual time=16.153..16.156 rows=6.00 loops=1)	
6	Group Key: t.id_sector, s.cod_sector	
7	Batches: 1 Memory Usage: 32kB	
8	Buffers: shared hit=2 read=40	
9	-> Hash Join (cost=2.91..1096.87 rows=46054 width=15) (actual time=0.081..9.361 rows=45511.00 loops=1)	
10	Hash Cond: (t.id_sector = s.id_sector)	
11	Buffers: shared hit=2 read=40	
12	-> Index Only Scan using idx_telemetria_sector on evento_telemetria t (cost=0.29..966.24 rows=46054 width=8) (actual time=0.042..3.107 rows=45511.00 lo...	
13	Index Cond: (id_partida = 1)	
14	Heap Fetches: 0	
15	Index Searches: 1	
16	Buffers: shared hit=1 read=40	
17	-> Hash (cost=1.72..1.72 rows=72 width=11) (actual time=0.034..0.034 rows=72.00 loops=1)	
18	Buckets: 1024 Batches: 1 Memory Usage: 12kB	
19	Buffers: shared hit=1	
20	-> Seq Scan on sector s (cost=0.00..1.72 rows=72 width=11) (actual time=0.010..0.018 rows=72.00 loops=1)	
21	Buffers: shared hit=1	
22	Planning:	
23	Buffers: shared hit=23 read=1	
24	Planning Time: 0.360 ms	
25	Execution Time: 16.210 ms	

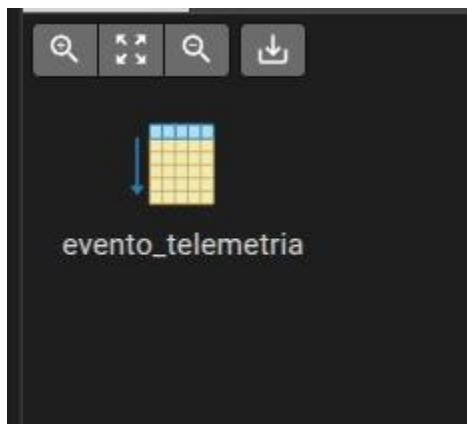


En esta Consulta se comparó el rendimiento al identificar los sectores más visitados dentro de una partida.

4.4.3 Consulta espacial por coordenadas.

Antes:

	QUERY PLAN
	text
1	Seq Scan on evento_telemetria (cost=0.00..3828.88 rows=126 width=28) (actual time=3.664..10.965 rows=2014.00 loo...
2	Filter: (point(posicion_x, posicion_y) <@ '(250,0),(0,-250)::box)
3	Rows Removed by Filter: 123978
4	Buffers: shared hit=1939
5	Planning:
6	Buffers: shared hit=3
7	Planning Time: 0.429 ms
8	Execution Time: 11.069 ms



Después:

	QUERY PLAN
	text
1	Bitmap Heap Scan on evento_telemetria (cost=9.26..410.21 rows=126 width=28) (actual time=0.376..0.746 rows=2014.00 loops=1)
2	Recheck Cond: (point(posicion_x, posicion_y) <@ '(250,0),(0,-250)::box)
3	Heap Blocks: exact=89
4	Buffers: shared hit=89 read=28
5	-> Bitmap Index Scan on idx_telemetria_espacial (cost=0.00..9.22 rows=126 width=0) (actual time=0.350..0.350 rows=2014.00 loo...
6	Index Cond: (point(posicion_x, posicion_y) <@ '(250,0),(0,-250)::box)
7	Index Searches: 1
8	Buffers: shared read=28
9	Planning:
10	Buffers: shared hit=18
11	Planning Time: 1.330 ms
12	Execution Time: 0.855 ms

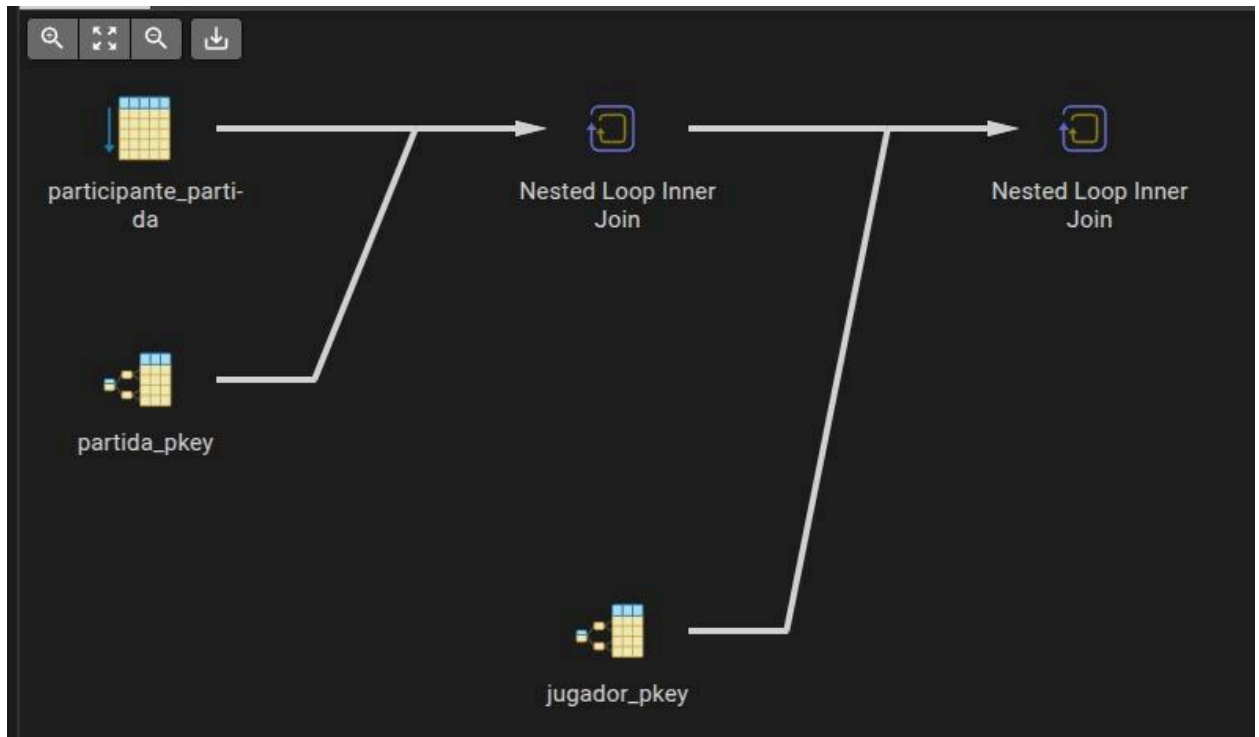


En esta consulta se evaluó el rendimiento al buscar eventos de telemetría dentro de un rango espacial usando las coordenadas X y Y

4.4.4 Consulta de participación de jugadores en partidas.

Antes:

	QUERY PLAN
	text
1	Nested Loop (cost=0.30..17.45 rows=1 width=68) (actual time=0.313..0.317 rows=1.00 loops=1)
2	Buffers: shared hit=5
3	-> Nested Loop (cost=0.15..9.27 rows=1 width=40) (actual time=0.184..0.186 rows=1.00 loops=1)
4	Buffers: shared hit=3
5	-> Seq Scan on participante_partida pp (cost=0.00..1.07 rows=1 width=8) (actual time=0.096..0.098 rows=1.00 loops=1)
6	Filter: (id_jugador = 3)
7	Rows Removed by Filter: 5
8	Buffers: shared hit=1
9	-> Index Scan using partida_pkey on partida p (cost=0.15..8.17 rows=1 width=36) (actual time=0.085..0.085 rows=1.00 loops=1)
10	Index Cond: (id_partida = pp.id_partida)
11	Index Searches: 1
12	Buffers: shared hit=2
13	-> Index Scan using jugador_pkey on jugador j (cost=0.15..8.17 rows=1 width=36) (actual time=0.128..0.129 rows=1.00 loops=1)
14	Index Cond: (id_jugador = 3)
15	Index Searches: 1
16	Buffers: shared hit=2
17	Planning:
18	Buffers: shared hit=99 dirtied=1
19	Planning Time: 1.818 ms
20	Execution Time: 0.336 ms



Después:

	QUERY PLAN	
	text	🔒
1	Nested Loop (cost=0.30..17.45 rows=1 width=68) (actual time=0.029..0.033 rows=1.00 loops=1)	
2	Buffers: shared hit=5	
3	-> Nested Loop (cost=0.15..9.27 rows=1 width=40) (actual time=0.024..0.026 rows=1.00 loops=1)	
4	Buffers: shared hit=3	
5	-> Seq Scan on participante_partida pp (cost=0.00..1.07 rows=1 width=8) (actual time=0.013..0.015 rows=1.00 loops=1)	
6	Filter: (id_jugador = 3)	
7	Rows Removed by Filter: 5	
8	Buffers: shared hit=1	
9	-> Index Scan using partida_pkey on partida p (cost=0.15..8.17 rows=1 width=36) (actual time=0.007..0.007 rows=1.00 loops=1)	
10	Index Cond: (id_partida = pp.id_partida)	
11	Index Searches: 1	
12	Buffers: shared hit=2	
13	-> Index Scan using jugador_pkey on jugador j (cost=0.15..8.17 rows=1 width=36) (actual time=0.004..0.005 rows=1.00 loops=1)	
14	Index Cond: (id_jugador = 3)	
15	Index Searches: 1	
16	Buffers: shared hit=2	
17	Planning:	
18	Buffers: shared hit=23 read=1	
19	Planning Time: 0.645 ms	
20	Execution Time: 0.064 ms	



En esta consulta se comparó el rendimiento al consultar la relación entre jugadores y partidas mediante la tabla “participante_partida”.

4.5 Creación de Vistas

Con el objetivo de simplificar consultas recurrentes y facilitar el acceso a información consolidada, se implementaron vistas sobre los datos más relevantes del sistema.

Las vistas permiten encapsular consultas complejas y presentarlas como si fueran tablas convencionales, facilitando su reutilización y mejorando la legibilidad de futuras consultas.

Vista 1

Descripción: La vista **vista_resumen** resume la información de telemetría de cada jugador en cada partida. Permite ver cuántos registros de telemetría tuvo cada jugador, cuál fue su primer y último tic registrado, cuánto duró su recorrido en tics y cuáles fueron sus promedios de salud y armadura durante la partida. Esta vista es útil para analizar la participación de los jugadores dentro de cada partida y comparar la duración de sus recorridos.

Código SQL:

18
is (6)
.A
3l 6

ts
alogs
nt Trigg
ensions
sign Data
guages
lications
emas
scription
telemetr
ts
alogs
nt Trigg

No limit

Query Query History

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

```
CREATE OR REPLACE VIEW vista_resumen AS
SELECT
    et.id_partida,
    et.id_jugador,
    j.nickname,
    COUNT(*) AS total_tics,
    MIN(et.tic) AS primer_tic,
    MAX(et.tic) AS ultimo_tic,
    MAX(et.tic) - MIN(et.tic) AS duracion_tics,
    AVG(et.salud) AS salud_promedio,
    AVG(et.armadura) AS armadura_promedio
FROM evento_telemetria et
JOIN jugador j
    ON j.id_jugador = et.id_jugador
GROUP BY
    et.id_partida,
    et.id_jugador,
    j.nickname;
```

Resultado:

79
80
81
82
83

```
SELECT * FROM vista_resumen
LIMIT 10;
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 6 Page No:

	id_partida Integer	id_jugador Integer	nickname text	total_tics bigint	primer_tic integer	ultimo_tic integer	duracion_tics integer	salud_promedio numeric	armadura_promedio numeric
1	1	1	DoomSlayer_1	23282	188	25271	25083	85.6382613177562065	91.7100764539128941
2	1	2	DoomSlayer_2	22229	175	28754	28579	100.8420981600611813	74.5922443654685321
3	2	3	DoomSlayer_3	25213	188	34784	34596	84.1930750009915520	27.0418831555150121
4	2	4	DoomSlayer_4	18656	188	23398	23210	75.9223842195540309	28.9674099485420240
5	3	5	DoomSlayer_5	30477	188	36421	36233	65.3470157823932802	51.4710437379007120
6	3	6	DoomSlayer_6	6135	188	6658	6470	84.9147514262428688	0.000000000000000000

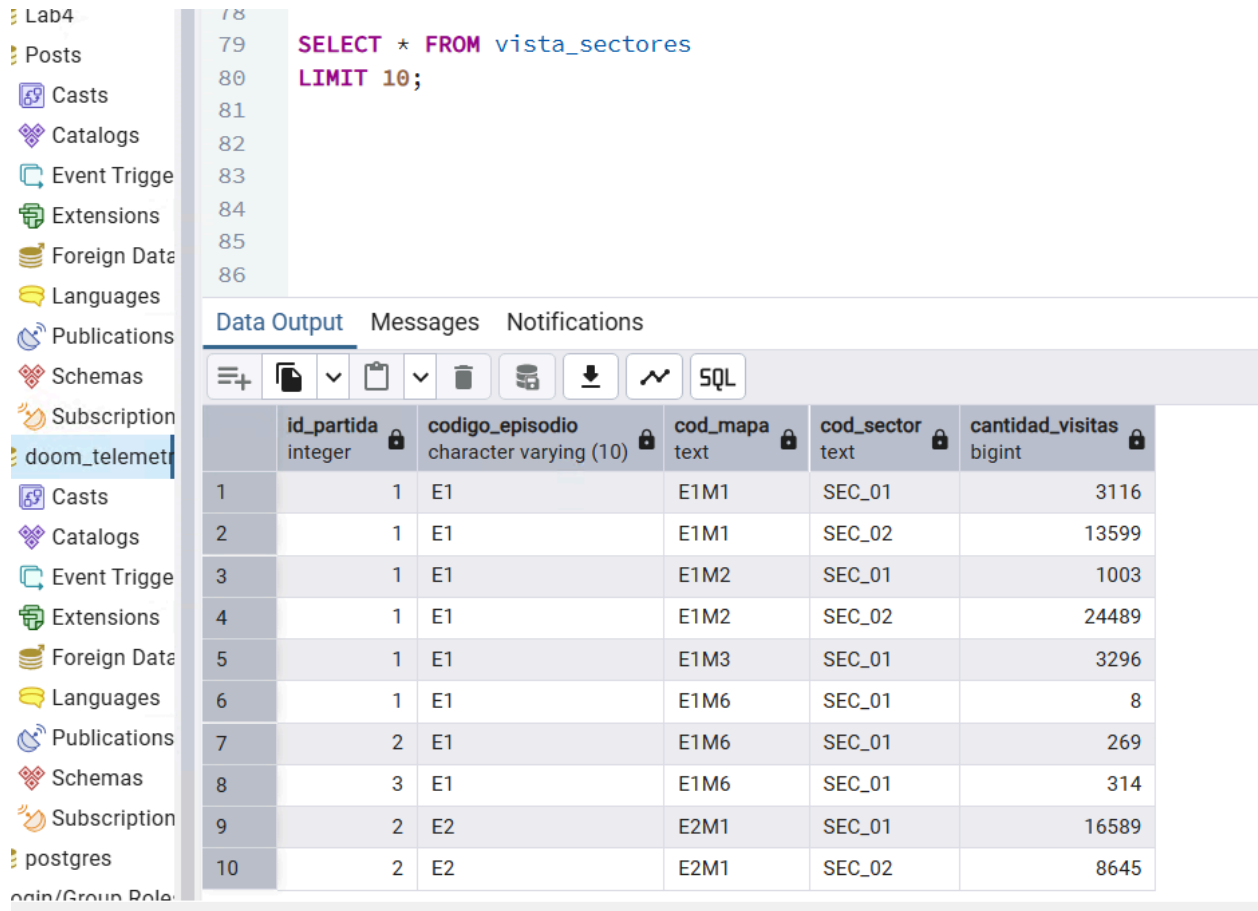
Vista 2

Descripción: La vista **vista_sectores** resume la cantidad de veces que los jugadores estuvieron en cada sector del mapa, agrupando la información por partida, episodio, mapa y sector. Esta vista permite identificar los sectores más visitados dentro de cada mapa. Es útil para encontrar zonas de mayor actividad o puntos calientes dentro del juego.

Código SQL:

	Query	Query History
28		
29	CREATE OR REPLACE VIEW vista_sectores AS	
30	SELECT	
31	et.id_partida,	
32	e.codigo_episodio,	
33	m.cod_mapa,	
34	s.cod_sector,	
35	COUNT (*) AS cantidad_visitas	
36	FROM evento telemetria et	
37	JOIN sector s	
38	ON s.id_sector = et.id_sector	
39	JOIN mapa m	
40	ON m.id_mapa = s.id_mapa	
41	JOIN episodio e	
42	ON e.id_episodio = m.id_episodio	
43	GROUP BY	
44	et.id_partida,	
45	e.codigo_episodio,	
46	m.cod_mapa,	
47	s.cod_sector;	
48		

Resultado:



The screenshot shows a database management interface. On the left is a sidebar with a tree view containing items like Lab4, Posts, Casts, Catalogs, Event Triggers, Extensions, Foreign Data, Languages, Publications, Schemas, Subscription, doom_telemet, Casts, Catalogs, Event Triggers, Extensions, Foreign Data, Languages, Publications, Schemas, Subscription, postgres, and a partially visible 'Login/Group Role'. The main area displays a SQL query in a text editor:

```
78  
79 SELECT * FROM vista_sectores  
80 LIMIT 10;  
81  
82  
83  
84  
85  
86
```

Below the editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table of results. Above the table is a toolbar with icons for expand, save, undo, redo, delete, refresh, download, and a button labeled 'SQL'. The table has the following structure:

	id_partida integer	codigo_episodio character varying (10)	cod_mapa text	cod_sector text	cantidad_visitas bigint
1	1	E1	E1M1	SEC_01	3116
2	1	E1	E1M1	SEC_02	13599
3	1	E1	E1M2	SEC_01	1003
4	1	E1	E1M2	SEC_02	24489
5	1	E1	E1M3	SEC_01	3296
6	1	E1	E1M6	SEC_01	8
7	2	E1	E1M6	SEC_01	269
8	3	E1	E1M6	SEC_01	314
9	2	E2	E2M1	SEC_01	16589
10	2	E2	E2M1	SEC_02	8645

4.6 Creación de Vista Materializada

Además de las vistas convencionales, se implementó una vista materializada para almacenar físicamente resultados de consultas complejas y reducir el tiempo de respuesta en análisis frecuentes.

La vista materializada resulta especialmente útil cuando los datos consultados cambian con poca frecuencia pero son utilizados constantemente para generación de reportes.

Descripción:

La vista materializada **vista_mate_jugador** guarda un resumen general de cada jugador. Muestra

cuántos registros de telemetría tiene cada jugador, en cuántas partidas participó, su salud promedio, su armadura promedio, su primer tic registrado y su último tic registrado.

A diferencia de una vista normal, esta vista materializada guarda físicamente el resultado de la consulta. Por eso puede ser útil para análisis frecuentes, ya que permite consultar más rápido la información resumida por jugador.

Si se cargan nuevos datos en la tabla evento_telemetria, la vista materializada debe actualizarse con:

REFRESH MATERIALIZED VIEW vista_mate_jugador;

Código SQL:

```
60
61 CREATE MATERIALIZED VIEW vista_mate_jugador AS
62 SELECT
63     et.id_jugador,
64     j.nickname,
65     COUNT(*) AS total_eventos,
66     COUNT(DISTINCT et.id_partida) AS total_partidas,
67     AVG(et.salud) AS salud_promedio,
68     AVG(et.armadura) AS armadura_promedio,
69     MIN(et.tic) AS primer_tic,
70     MAX(et.tic) AS ultimo_tic
71 FROM evento_telemetria et
72 JOIN jugador j
73     ON j.id_jugador = et.id_jugador
74 GROUP BY
75     et.id_jugador,
76     j.nickname;
```


Resultado:

78
79
80
81
82
83
84
85
86

SELECT * FROM vista_mate_jugador

Data Output Messages Notifications									
Showing rows: 1 to 6									
	id_jugador integer	nickname text	total_eventos bigint	total_partidas bigint	salud_promedio numeric	armadura_promedio numeric	primer_tic integer	ultimo_tic integer	
1	1	DoomSlayer_1	23282	1	85.6382613177562065	91.7100764539128941	188	25271	
2	2	DoomSlayer_2	22229	1	100.8420981600611813	74.5922443654685321	175	28754	
3	3	DoomSlayer_3	25213	1	84.1930750009915520	27.0418831555150121	188	34784	
4	4	DoomSlayer_4	18656	1	75.9223842195540309	28.9674099485420240	188	23398	
5	5	DoomSlayer_5	30477	1	65.3470157823932802	51.4710437379007120	188	36421	
6	6	DoomSlayer_6	6135	1	84.9147514262428688	0.000000000000000000	188	6658	

4.7 Resultado de la Tercera Entrega

Como resultado de esta fase se logró transformar los datos almacenados en información analítica útil para el estudio del comportamiento de los jugadores. La implementación de consultas especializadas permitió identificar patrones relevantes dentro de las partidas, mientras que los índices y vistas optimizaron significativamente el acceso a la información.

Además, el mecanismo de recreación garantiza la reproducibilidad del proyecto y facilita futuras extensiones o investigaciones basadas en la misma estructura de datos.

6. Análisis General

Los resultados obtenidos permitieron comprobar que la base de datos es capaz de almacenar grandes volúmenes de telemetría y relacionarlos con información de usuarios y encuestas UX. Las consultas analíticas permitieron identificar patrones de movimiento, permanencia en sectores, proximidad entre jugadores y posibles indicadores de cooperación dentro de las partidas.

Asimismo, la implementación de índices y vistas mejoró significativamente el rendimiento de las consultas, demostrando la importancia de la optimización en sistemas con grandes cantidades de datos.

7. CONCLUSIONES

El proyecto permitió aplicar de manera práctica los conceptos fundamentales de bases de datos relacionales, desde el modelado conceptual hasta la optimización de consultas sobre grandes conjuntos de datos. La implementación del proceso ETL demostró la importancia de contar con mecanismos eficientes para la integración y transformación de información proveniente de fuentes externas.

Asimismo, el uso de índices, vistas y consultas analíticas permitió mejorar significativamente el rendimiento del sistema y facilitar la obtención de información relevante para el análisis de comportamiento de los jugadores. La integración de datos de telemetría y encuestas UX proporcionó una visión más completa de la experiencia de juego y abrió la posibilidad de realizar estudios sobre cooperación, interacción y patrones de movimiento.

En conclusión, se logró construir una solución funcional, escalable y alineada con los objetivos planteados, evidenciando la utilidad de las bases de datos como herramienta para el almacenamiento, organización y análisis de información en entornos de investigación y analítica de videojuegos.